

EX NO: 01

DATE:

BASIC I/O PROGRAMMING

Aim:

TO write a Program to Counting the number of Vowels, Consonants, words, white space in a line of text.

ALGORITHM:

Step 1: Start the Program

Step 2: Set $S=0$, $w=1$, $C=0$, $V=0$

Step 3: Define String $Str = \text{" "}$

Step 4: Set $i=0$

Step 5: Repeat Step 5 to Step 7 until, $i < Str + 1$ length()

Step 6: IF any character of Str matches with any vowels
Then $V=V+1$

Step 7: IF any character of Str vowels line Between
a and $z <= C+1$

Step 8: $i = i + 1$

Step 9: Then include to number of word count= w and
no of white space= S

Step 10: Print V , C , w , S

Step 11: Stop the Program.

RESULT:

Thus the Program has been executed successfully.

Ex NO: 02

DATE:

FIRST COME FIRST SERVED ALGORITHM

AIM:

TO Simulate the CPU scheduling algorithm First come First Serve (FCFS).

ALGORITHM:

Step 1: Start the Process

Step 2: Access the number of Process in the ready Queue.

Step 3: For Each Process in the ready Q, assign the Process name and the burst time.

Step 4: Set the waiting of the first Process as '0' and its burst time as its turnaround time

Step 5: For each Process in the Ready a
Calculate

$$a) \text{ waiting time}(n) = \text{waiting time}(n-1) + \text{Burst time}(n-1)$$

$$b) \text{ Turnaround time}(n) = \text{waiting time}(n) + \text{Burst time}(n)$$

Step 6: calculate

$$a) \text{ Average waiting time} = \text{total waiting time} / \text{number of Process}$$

$$b) \text{ Average turnaround time} = \text{total turnaround time} / \text{number of Process}$$

Step 7: Stop the Process

RESULT:

Thus the Program has been executed successfully.

Ex NO: 03

DATE:

SHORTEST JOB FIRST ALGORITHM

AIM:

TO write a Program to simulate the CPU scheduling Algorithm Shortest job First (non)-Preemption).

ALGORITHM:

Step 1: Start the Process

Step 2: ACCEPT the number of Processes in the ready Queue.

Step 3: For each Process in the ready Q. assign the Process ID and accept the CPU burst time.

Step 4: Start the Ready Queue according to Shortest Burst time by Sorting according to lowest to highest burst time

Step 5: Set a waiting time of the first Process as '0' and its turnaround time as its burst time

Step 6: Sort the Process in the ready Queue

Step 7: Calculate

$$a) \text{ waiting time } (n) = \text{waiting time } (n-1) + \text{Burst time } (n-1)$$

$$b) \text{ Turnaround time } (n) = \text{waiting time } (n) + \text{Burst time } (n)$$

Step 8: Calculate

$$c) \text{ Average waiting Time} = \text{total waiting time} / \text{number of Process}$$

$$d) \text{ Average turnaround time} = \text{total turnaround time} / \text{number of Process}$$

Step 9: STOP the Process.

RESULT:

Thus the Program has been executed successfully.

Ex No: 01

DATE:

ROUND ROBIN ALGORITHM

AIM:

TO Simulate the CPU scheduling algorithm round robin

ALGORITHM:

Step 1: Start the Process

Step 2: Accept the number of Processes in the ready Queue and time quantum (or) time slice.

Step 3: For each Process in the ready Q, assign the Process and accept the CPU burst time.

Step 4: Calculate the no. of time slices for each Process where no. of time slice for Process (n) burst time Process (n) / time slice

Step 5: If the burst time is less than the time slice then the no. the time slice = 1.

Step 6: Consider the ready queue is a Circular Q, calculate
a) waiting time for Process (n) = waiting time for Process (n-1) + the time difference in getting the CPU from Process (n-1)

b) turnaround time for Process n = waiting time of Process (n) + burst time of Process (n) + the time difference in getting CPU from Process (n)

Step 7: calculate

a) Average waiting time = total waiting time / number of Process

b) Average Turnaround time = total turnaround time / number of Process

Step 8: Stop the Process

RESULT:

Thus the Program has been executed successfully.

Ex NO: 05

DATE:

PRIORITY SCHEDULING ALGORITHMS

AIM:

To write a Program to simulate the CPU Scheduling Priority algorithms.

ALGORITHM:

Step 1: Start the Program

Step 2: For each Process in the ready Q Accept the number of Processes in the ready Queue.

Step 3: For each Process in the ready Queue assign the Process id and accept the CPU burst time

Step 4: Sort the ready Queue according to the Priority number

Step 5: Set the waiting time First Process as '0' and its burst time as its turnaround time.

Step 6: Arrange the Process based on Process Priority

Step 7: For each Process in the ready Q calculate.

Step 8: For each Process in the Ready Q

a) waiting time (n) = waiting time (n-1) + Burst time (n-1)

b) Turnaround time (n) = waiting time (n) + Burst time (n)

Step 9: calculate

c) Average waiting time = total waiting time / number of Process

d) Average Turnaround time = total turn around time / number of

Process Print the results in Queue order

Step 10: Stop the Process

RESULT:

Thus the Program has been executed successfully.

Ex NO: 06

DATE:

READER / WRITER PROBLEM USING SEMAPHORE

AIM:

To implement reader/writer Problem using semaphore.

ALGORITHM:

Step 1: Start the Program.

Step 2: Declare the semaphore. Variable mutex and a shared variable which is used by reader and writer Process.

Step 3: wait and signal are two function to implement the semaphore.

wait - waits until semaphore variable reach 1 and then determine it

signal - increment the semaphore variable. by 1.

Step 4: The reader Process check if any - Process writing
If so it wait else if reads the context of shared variable and then signals.

Step 5: The writer Process checks if any other Process accessing the shared variable, If not it. change the value of shared variable and then signals.

Step 6: The words is used to execute two Process simultaneously.

Step 7: Stop the Program.

RESULT:

Thus the Program has been executed successfully.

Ex NO: 07	TO IMPLEMENT BANKER'S ALGORITHM FOR DEADLOCK AVOIDANCE
DATE:	

AIM:

TO simulate bankers algorithm for Dead lock Avoidance (Banker's) Algorithm.

ALGORITHM:

- Step 1: Start the Program.
- Step 2: Get the values of resources and Processes
- Step 3: Get the Quail value.
- Step 4: After allocation find the need value.
- Step 5: Check whether its possible to allocate
- Step 6: IF it is possible then the system is in safe state.
- Step 7: ELSE system is not in safety state.
- Step 8: IF the new request comes than check that the system is in safety.
- Step 9: OR NOT if we allow the request
- Step 10: STOP the Program.

RESULT:

Thus the Program has been executed successfully.

Ex No: 08

DATE:

FIRST IN FIRST OUT ALGORITHMS

AIM:

TO implement FIFO Page replacement technique.

ALGORITHM:

STEP 1: Start the Process

STEP 2: Read number of Pages no

STEP 3: Read number of Pages no

STEP 4: Read Page number into an array [a]

STEP 5: Initialize $Circle[i] = 0$ to check Page bit

STEP 6: Replace the page with circular queue, while replacing check Page availability in the Frame Place $Circle[i] = 1$ if Page is Placed in the Frame count Page Faults.

STEP 7: Print the results.

STEP 8: STOP the Process.

RESULT:

Thus the Program has been executed successfully.

Ex NO: 09

DATE:

LEAST RECENTLY USED ALGORITHM (LRU)

AIM:

TO implement Least Recently used (LRU) Page Replacement Language.

ALGORITHM:

- Step 1: Start the Process
- Step 2: Declare the size
- Step 3: Get the number of Pages to be inserted
- Step 4: Get the value.
- Step 5: Declare Counters and Stack
- Step 6: select the least recently used Page counters value
- Step 7: Stack them according the selection.
- Step 8: Display the value.
- Step 9: STOP the Process.

RESULT:

Thus the Program has been executed successfully.

Ex NO: 10(a)

DATE:

MEMORY MANAGEMENT ALGORITHM FIRST FIT

AIM:

To write a C Program for implementation of FCF8 and SIF scheduling algorithms.

ALGORITHM:

Step 1: Define the MAX as 25

Step 2: Declare the variable $Frog[Max]$, $b[Max]$, $F[Max]$, i, j, n, b , n, F , $temp$ highest = 0, $b[Max]$, $FF[Max]$

Step 3: Get the number of blocks, files, size of the blocks using for loop.

Step 4: In for loop check $bF[j] = 1$, if so $temp = b[j] - F[i]$

Step 5: Check $temp > temp$, if so assign $FF[i] = j$, highest $temp$

Step 6: Assign $Frog[i] = lowest$, $bF[FF[i]] = 1$, $lowest = 10000$

Step 7: Repeat Step 4 to Step 6

Step 8: Print file no, size, block no, size and fragment.

Step 9: Stop the Program.

RESULT:

Thus the Program has been executed successfully.

Ex No: 10(B)

DATE:

MEMORY MANAGEMENT ALGORITHM BEST FIT

AIM:

TO write a C Program for implementation memory allocation methods for Fixed Position using First Fit.

ALGORITHM:

Step 1: Define the max as 25

Step 2: Declare the variable $prog[num]$, $bf[max]$, $f[max]$, i, j , nb , nf , $temp$, $highest = 0$, $bf[max]$, $ff[max]$

Step 3: Get the number of blocks, files, size of the block using for loop.

Step 4: In for loop check $bf[j] = 1$, if so $temp = b[f] - f[i]$

Step 5: Check $highest < temp$, if so assign $ff[i] = j$, $highest = temp$

Step 6: Assign $prog[i] = highest$, $bf[ff[i]] = 1$, $highest = 0$

Step 7: Repeat Step 4 to Step 6

Step 8: Print file no, size, block no, size and fragment.

Step 9: Stop the Program.

RESULT:

Thus the Program has been executed successfully.

Ex NO: 10(c)

DATE:

MEMORY MANAGEMENT ALGORITHM WORST FIT

AIM:

To write a C Program for implementation of FCFS and SJF scheduling algorithms.

ALGORITHM:

Step 1: Define the max as 25

Step 2: Declare the variable $frag[max]$, $F[max]$, i , j , nb , nF , top , $height = 0$, $bi[max]$, $FF[max]$

Step 3: Get the number of blocks files size of the blocks using for loop

Step 4: In for loop check $bf[j] \neq 1$, if so $temp = bt[j] = F[i]$

Step 5: Check $temp > 0$, if so assign $FF[i] = i$ break the for loop

Step 6: Assign $frag[i] = temp$, $bf[FF[i]] = 1$;

Step 7: Repeat Step 4 to Step 6

Step 8: Print file no, size, block no, size and fragment

Step 9: Stop the Program.

RESULT:

Thus the Program has been executed successfully.

Ex NO: 11

DATE:

INTER PROCESS COMMUNICATION (IPC)

AIM:

To implement the concept of interProcess Communication using C languages.

ALGORITHM:

Step 1: Create the pipe and create the Process.

Step 2: Get the input is the main Process and Pass the output to the child Process using Pipe.

Step 3: Perform the ~~Q~~ operation Given in the child Process and Print the output.

Step 4: STOP the Program.

RESULT:

Thus the Program has been executed successfully.